The first one uses the DOM ID, the second uses the form name, and the third uses the label text.

By default, Capybara matches on a substring of the name or label, not just the entire string, so you could also use “em” as a matcher should you want to. If more than one element matches a locator, Capybara raises an error. If you want an exact match rather than a substring match, pass the option exact: true to any Capybara method that uses a locator.

Which lookup text should you use? It depends on your goals and your context. The label text will usually result in the most readable test. But it’s also the most fragile since the user-facing text is most likely to change. In contrast, the DOM ID is probably the most opaque in the test but the least likely to change on a whim.

Here are the Capybara form-interaction methods.[3] First, the ones you’ll probably use a lot:

check(locator)

This asserts that locator finds a check box, and checks it.

choose(locator)

This is the same as check, but for radio buttons. Other radio buttons in the group are unchecked.

fill_in(locator, with: "TEXT")

The locator is expected to find a text area or an input text field, and places the value of the with option inside it. Technically the second argument is a generic options hash, but the only option that is unique to this method is with. There are some other options you can pass that are common to all these methods, like the exact option I talked about a moment ago.

select(value, from: locator)

The method arguments are similar here in that the second argument is technically an options hash, but the only usable option is from. It looks for a select menu that matches the locator passed to the from argument, and sets its value to the first argument. For what it’s worth, the fact that fill_in takes its locator argument first and select takes the locator second drives me bananas.

click_on(locator)

Finds an anchor link or button via the locator and simulates clicking it.

3. http://www.rubydoc.info/github/teamcapybara/capybara/master